实验四　Spring IoC-xml 配置

一、实验目的及任务

通过该实验掌握利用 Spring xml 配置方式实现控制反转 IoC(依赖注入)，掌握 Spring 的 Ioc 注入方式：属性注入和构造注入；。

二、实验环境及条件

主机操作系统为 Windows11，Tomcat 8 ,jdk1.8 版本。

三、实验实施步骤

略

四、实验结果

1 运行四、五、六步骤，分别给出完整程序源代码和控制台结果截图。

四：

1) 程序源代码

sdjzu.springIoc.Person.java

```
package sdjzu.springIoc;
public class Person {
    private String name;

    public void sayHello(){
        System.out.println("hello, i am " + name);
    }
    //省略get set 方法,请自行添加
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
}
```

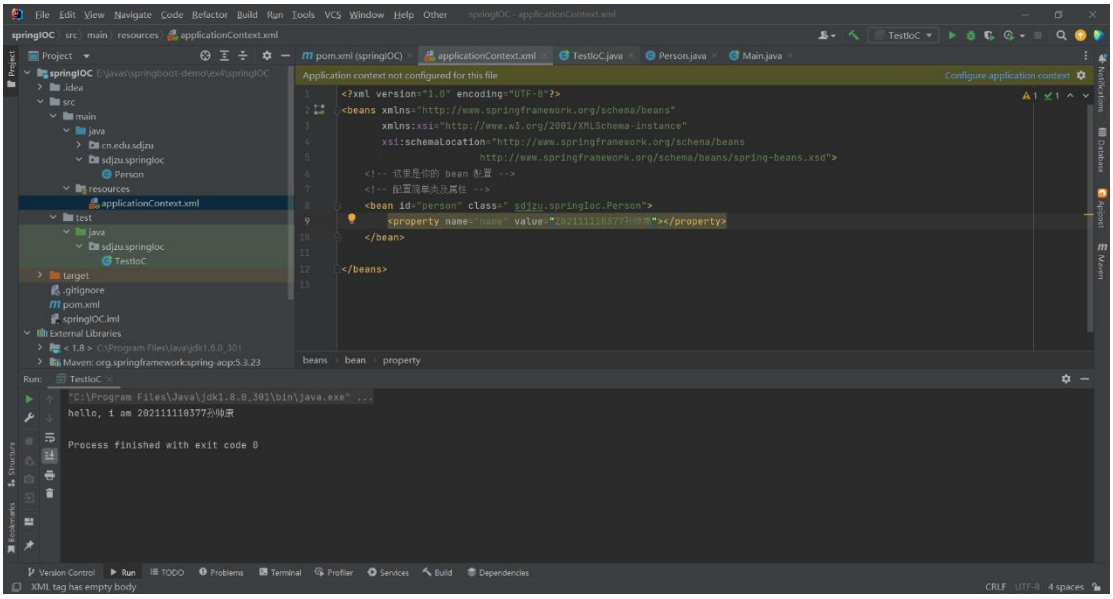
sdjzu.springIoc.TestIoC.java

```
package sdjzu.springIoc;

import org.springframework.context.ApplicationContext;
import
org.springframework.context.support.ClassPathXmlApplicationContext;

public class TestIoC{

    public static void main(String[] args) {
        ApplicationContext apc = new
ClassPathXmlApplicationContext("applicationContext.xml");

        Person person = (Person) apc.getBean("person");
        person.sayHello();
    }
}
```

applicationContext.xml

```
<!-- 配置简单类及属性 -->
<bean id="person" class=" sdjzu.springIoc.Person">
    <property name="name" value="202111110377 孙帅康"></property>
</bean>
```

2) 控制台结果



五：

1) 程序源代码

propertyInject.IPerson.java

```java
package propertyInject;
public interface IPerson {
    public void useAxe();
}
```

propertyInject.IAxe.jva

```java
package propertyInject;
public interface IAxe {
    public void chop();
}
```

propertyInject.American.java

```java
package propertyInject;
public class American implements IPerson {
    private IAxe axe;

    public IAxe getAxe() {
        return axe;
    }

    public void setAxe(IAxe axe) {
        this.axe = axe;
    }

    public void useAxe() {
        axe.chop();
    }
}
```

propertyInject.Chinese.java

```java
package propertyInject;
public class Chinese implements IPerson {
    private IAxe axe;

    public IAxe getAxe() {
        return axe;
    }

    public void setAxe(IAxe axe) {
        this.axe = axe;
    }

    public void useAxe() {
        axe.chop();
    }
}
```

propertyInject.SteelAxe.java

```
package propertyInject;

public class SteelAxe implements IAxe {
    public void chop() {
        System.out.println("ssk:steelAxe is sharp!!!");
    }
}
```

propertyInject.StoneAxe.java

```
package propertyInject;

public class StoneAxe implements IAxe {
    public void chop() {
        System.out.println("ssk:stoneAxe is blunt!!!");
    }
}
```

applicationContext.xml

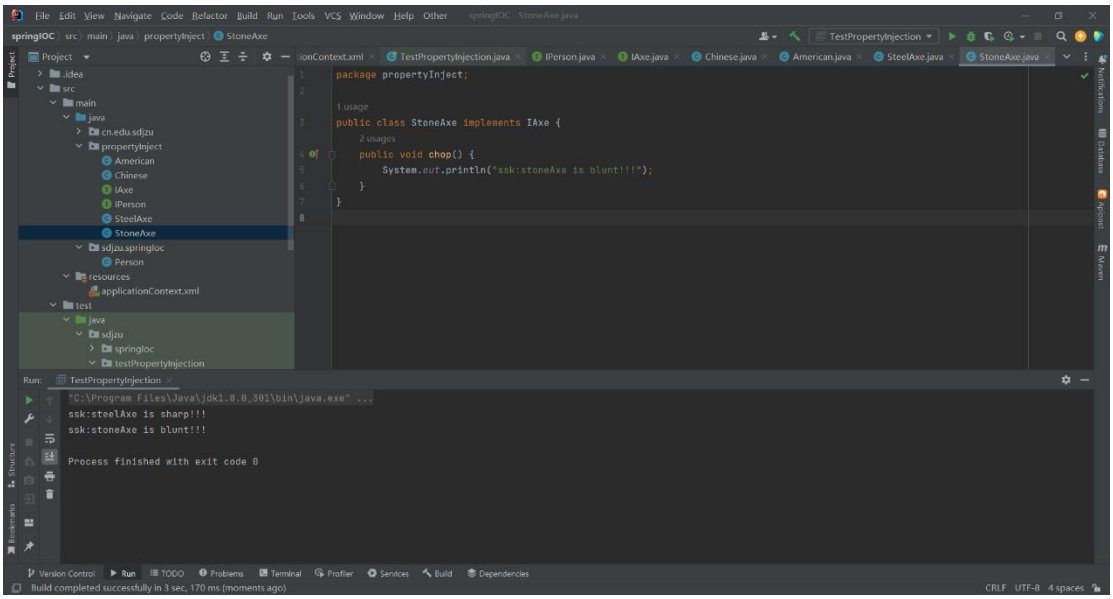
```
<!--配置类及使用 bean -->
<bean id="stoneAxe" class="propertyInject.StoneAxe"></bean>
<bean id="steelAxe" class="propertyInject.SteelAxe"></bean>

<bean id="chinese" class="propertyInject.Chinese">
    <property name="axe" ref="steelAxe"></property>
</bean>
<bean id="american" class="propertyInject.American">
    <property name="axe" ref="stoneAxe"></property>
</bean>
```

sdjzu.testPropertyInjection.TestPropertyInjection.java

```
package sdjzu.testPropertyInjection;

import org.springframework.context.ApplicationContext;
import
org.springframework.context.support.ClassPathXmlApplicationContext;
import propertyInject.American;
import propertyInject.Chinese;

public class TestPropertyInjection{

    public static void main(String[] args) {

        ApplicationContext apc = new
ClassPathXmlApplicationContext("applicationContext.xml");
```

```
        //second 配置类及使用 bean

        Chinese chinese = (Chinese) apc.getBean("chinese");
        chinese.useAxe();

        American american = (American) apc.getBean("american");
        american.useAxe();

    }
}
```

2) 控制台结果



六：

1) 程序源代码

sdjzu.constructorInject.France.java

```
package sdjzu.constructorInject;

import propertyInject.IAxe;
import propertyInject.IPerson;

public class France implements IPerson {
    private IAxe axe;
    private IAxe axe2;
    private String name;
    public France() {
    }
    public France(IAxe axe,IAxe axe2,String name) {
```

```
        this.axe = axe;
        this.axe2 = axe2;
        this.name = name;
    }
    public void useAxe() {
        axe.chop();
        axe2.chop();
        System.out.println("在 useAxe 内显示: "+name);
    }
}
```

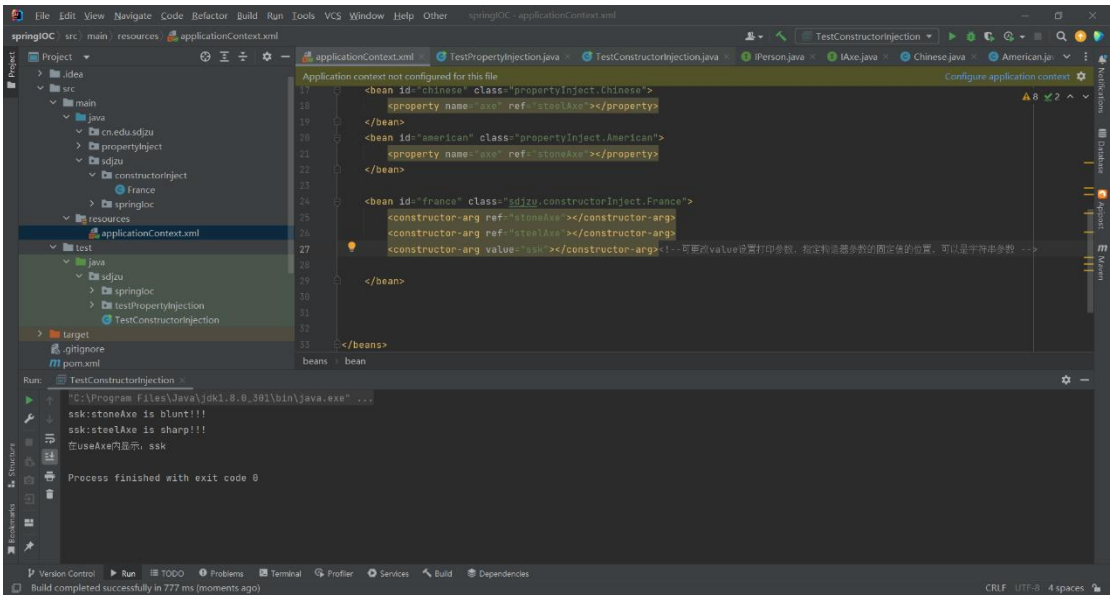
applicationContext.xml

```
<bean id="france" class="sdjzu.constructorInject.France">
    <constructor-arg ref="stoneAxe"></constructor-arg>
    <constructor-arg ref="steelAxe"></constructor-arg>
    <constructor-arg value="ssk"></constructor-arg><!--可更改 value 设置打印参数，指定构造器参数的固定值的位置，可以是字符串参数 -->

</bean>
```

sdjzu.TestConstructorInjection.java

```
package sdjzu;

import org.springframework.context.ApplicationContext;
import
org.springframework.context.support.ClassPathXmlApplicationContext;
import sdjzu.constructorInject.France;

public class TestConstructorInjection{

    public static void main(String[] args) {
        ApplicationContext apc = new
ClassPathXmlApplicationContext("applicationContext.xml");

        //构造注入
        France france = (France) apc.getBean("france");
        france.useAxe();

    }
}
```

2) 控制台结果



2 你遇到了什么问题，如何解决的？

在本次实验中，过程较为顺利，掌握了利用 Spring xml 配置方式实现控制反转 IoC(依赖注入)，掌握 Spring 的 Ioc 注入方式：属性注入和构造注入。以及注入普通类、使用 bean 设值属性注入和构造注入，收获满满。